

Guía de ejecución

Cómo poner en marcha StockFlow, paso a paso

StockFlow — Inventario, compras internas y movimientos de almacén
(PA-06)

Docker Desktop · PostgreSQL 16 · Spring Boot 3.5 · IntelliJ IDEA ·
DataGrip
React 19 · Vite 8 · WebStorm

Para quién	Cualquier integrante del equipo, sin experiencia previa en Docker, Spring Boot ni React
Sistema	Linux (CachyOS) con Docker Desktop, Java 21, Node.js y JetBrains Toolbox
Tiempo	Primera vez: 20-30 minutos · Después: menos de 3 minutos
Fecha	5 de octubre de 2026 · rama Hector (incluye el frontend)

Los mensajes de consola que aparecen en esta guía se copiaron de un arranque real del proyecto.

Índice

1. Antes de empezar	3
2. Preparación (se hace una sola vez)	5
Preparación 1 · Comprobar que tienes Java 21	5
Preparación 2 · Que el comando docker hable con Docker Desktop	5
Preparación 3 · Comprobar que tienes Node.js y npm	5
Preparación 4 · Tener el proyecto actualizado	6
3. Poner en marcha la base y el backend, paso a paso	7
Paso 1 · Encender Docker Desktop	7
Paso 2 · Levantar la base de datos	7
Paso 3 · Comprobar que la base está lista	8
Paso 4 · Abrir el backend en IntelliJ IDEA	8
Paso 5 · Elegir Java 21 en IntelliJ (sólo la primera vez)	9
Paso 6 · Ejecutar StockFlowApplication	9
Paso 7 · Comprobar en el navegador que el backend responde	10
Paso 8 · Probar la API con Swagger UI	10
Paso 9 · Probar todos los casos con requests.http	11
Paso 10 · Ver el SQL que ejecuta Hibernate	11
Paso 11 · Ver los datos en DataGrip	12
Paso 12 · Ejecutar las pruebas automáticas	12
4. Poner en marcha el frontend (React), paso a paso	13
Frontend 1 · Abrir el frontend en WebStorm	13
Frontend 2 · Instalar las dependencias (la primera vez)	13
Frontend 3 · Revisar la dirección del backend (.env.development)	14
Frontend 4 · Encender el frontend	14
Frontend 5 · Abrir la aplicación en el navegador	15
Frontend 6 · Probar el CRUD desde las pantallas	15
Frontend 7 · Ejecutar las pruebas y el build del frontend	16
5. Cómo apagar todo	17
6. Rutina rápida (para los días siguientes)	17
7. Problemas frecuentes y cómo resolverlos	19
8. Glosario	21

1. Antes de empezar

Esta guía te lleva, paso a paso, desde la computadora apagada hasta tener **StockFlow funcionando**: la base de datos encendida, el backend respondiendo, la API probada, los datos visibles en DataGrip y la **aplicación web (frontend)** abierta en el navegador.

No necesitas saber Docker, Spring Boot ni React para seguirla. Cada paso dice **qué hacer**, el **comando exacto** para copiar, **qué deberías ver** si salió bien y **qué hacer si falla**.

Hazla en orden la primera vez: primero la base de datos y el backend (capítulo 3) y después el frontend (capítulo 4), porque el frontend **necesita** al backend encendido para mostrar datos. Después, para el uso diario, alcanza con la **rutina rápida** del capítulo 6.

Cómo leer cada paso

Los bloques oscuros son comandos: cópialos tal cual en la terminal (o direcciones para pegar en el navegador).

✓ Qué deberías ver

Lo que aparece cuando el paso salió bien.

⚠ Si falla

Qué hacer si ves otra cosa.

i Para entenderlo

Por qué se hace así (opcional, pero útil para la defensa).

Las piezas del sistema

Pieza	Qué es (en simple)	Dónde vive
Docker Desktop	Un programa que ejecuta otros programas dentro de "cajas" aisladas llamadas contenedores. Lo usamos para no instalar PostgreSQL directamente en tu sistema.	Se enciende con <code>systemctl --user start docker-desktop</code>
PostgreSQL	La base de datos: guarda categorías, productos, stock, movimientos...	Dentro del contenedor <code>stockflow-postgres</code> , en <code>localhost:5432</code>
Backend (Spring Boot)	El programa Java que recibe las peticiones, aplica las reglas del negocio y lee o escribe en PostgreSQL.	Se ejecuta desde IntelliJ (<code>StockFlowApplication</code>), en <code>localhost:8080</code>
Flyway	Una herramienta del backend que crea las tablas automáticamente al arrancar, usando los archivos V1, V2 y V3.	<code>src/main/resources/db/migration</code>
Frontend (React + Vite)	La aplicación web con pantallas: menú, tablas y formularios para categorías y productos. No guarda datos: se los pide al backend.	<code>webstorm/stockflow-frontend</code> ; se ejecuta con <code>npm run dev</code> , en <code>localhost:5173</code>
Node.js y npm	Node.js ejecuta las herramientas del frontend (Vite, las pruebas). npm descarga sus librerías, como Maven en el backend.	Instalados en tu sistema (<code>node</code> , <code>npm</code>)

Pieza	Qué es (en simple)	Dónde vive
Cientes de prueba	Otros programas con los que pruebas el backend directamente, sin el frontend: Swagger UI, <code>requests.http</code> , Postman. DataGrip mira la base.	Tu navegador, IntelliJ, Postman y DataGrip

Cómo se conectan: Tú, en el navegador → **Frontend** en localhost:5173 (WebStorm, `npm run dev`) → **Backend** en localhost:8080 (IntelliJ) → **PostgreSQL** en localhost:5432 (contenedor Docker) ← DataGrip (para mirar las tablas). Swagger, `requests.http` y Postman hablan directo con el backend.

2. Preparación (se hace una sola vez)

Preparación 1 · Comprobar que tienes Java 21

Abre una terminal (Konsole o la que uses) y escribe:

```
java -version
```

✓ Qué deberías ver

Una línea que empiece con `openjdk version "21.0.12.1"`. En tu computadora: `openjdk version "21.0.12.1"`.

⚠ Si falla

Si dice `command not found` o una versión distinta de 21, instala Java 21: `sudo pacman -S jdk21-openjdk`.

Preparación 2 · Que el comando docker hable con Docker Desktop

Docker Desktop crea su propio "canal" de comunicación (se llama *contexto*). Hay que decirle al comando `docker` que use ese canal. **Se hace una sola vez**; en tu computadora ya está hecho.

Para comprobarlo:

```
docker context show
```

✓ Qué deberías ver

```
desktop-linux
```

⚠ Si falla

Si dice `default`, ejecuta `docker context use desktop-linux` y vuelve a comprobar.

i Para entenderlo

El contexto `default` busca a Docker en `/var/run/docker.sock`, que en tu sistema no existe porque tienes Docker Desktop y no Docker Engine. Por eso `sudo systemctl start docker` responde `Unit docker.service not found`: ese servicio no está instalado.

Preparación 3 · Comprobar que tienes Node.js y npm

El frontend necesita **Node.js** (mínimo 20.19) y **npm**. En la terminal:

```
node -v  
npm -v
```

✓ Qué deberías ver

Dos números de versión. En tu computadora: `v26.10.0` y `12.2.0`.

⚠ Si falla

Si dice `command not found`, instálalos con `sudo pacman -S nodejs npm` y vuelve a comprobar.

i Para entenderlo

Node.js es a JavaScript lo que Java es al backend: el programa que ejecuta las herramientas. npm es el equivalente a Maven: lee `package.json` y descarga las librerías (React, Vite, Vitest...).

Preparación 4 · Tener el proyecto actualizado

Entra a la carpeta del proyecto y comprueba en qué rama estás:

```
cd ~/Desktop/StockFlow
git status -sb
```

✓ Qué deberías ver

`## Hector...origin/Hector` sin la palabra `behind`.

⚠ Si falla

Si dice `behind`, descarga lo último con `git pull`.

3. Poner en marcha la base y el backend, paso a paso

Sigue los pasos en orden. Del 1 al 7 dejan la base y el backend funcionando; del 8 al 12 sirven para probar la API y para mostrar el proyecto en la defensa. El frontend viene en el capítulo 4.

Paso 1 · Encender Docker Desktop

En la terminal:

```
systemctl --user start docker-desktop
```

✓ Qué deberías ver

No muestra nada (eso es bueno). Docker Desktop tarda unos 20-60 segundos en estar listo. Puedes comprobarlo con `docker ps`: si muestra una tabla (aunque esté vacía), ya está listo.

⚠ Si falla

Si `docker ps` dice *failed to connect to the docker API*, espera unos segundos más y repite. Si sigue fallando, abre la aplicación **Docker Desktop** desde el menú de aplicaciones.

i Para entenderlo

Va **sin sudo** porque Docker Desktop es un servicio de **tu usuario** (`--user`), no del sistema.

Paso 2 · Levantar la base de datos

Desde la carpeta del proyecto:

```
cd ~/Desktop/StockFlow
docker compose up -d
```

✓ Qué deberías ver

Container `stockflow-postgres` `Started` (o `Running` si ya estaba encendido).

⚠ Si falla

Si dice que no encuentra el archivo, no estás en la carpeta correcta: revisa el `cd`.

i Para entenderlo

`docker compose` lee el archivo `docker-compose.yml` y crea un contenedor con PostgreSQL 16, la base `stockflow` y el usuario `stockflow_admin`. La opción `-d` lo deja corriendo en segundo plano y te devuelve la terminal.

Paso 3 · Comprobar que la base está lista

Espera unos 10 segundos y ejecuta:

```
docker ps
```

✓ Qué deberías ver

Una fila `stockflow-postgres` con el estado `Up ... (healthy)` y el puerto `0.0.0.0:5432->5432/tcp`.

⚠ Si falla

Si dice `(health: starting)`, espera unos segundos más y repite. Si no aparece la fila, repite el paso 2.

i Para entenderlo

`healthy` significa que el contenedor no sólo arrancó, sino que PostgreSQL ya acepta conexiones (lo comprueba el `healthcheck` del `docker-compose.yml`).

Paso 4 · Abrir el backend en IntelliJ IDEA

Muy importante: hay que abrir la carpeta `intellij/stockflow-backend`, **no** la carpeta `StockFlow`. Es la que contiene el archivo `pom.xml`; si abres otra, IntelliJ no reconoce el proyecto y no aparece el botón para ejecutarlo.

Forma A, desde la terminal:

```
~/local/share/JetBrains/Toolbox/scripts/idea \
~/Desktop/StockFlow/intellij/stockflow-backend
```

(Es un solo comando en dos líneas: la `\` final le dice a la terminal que continúa en la línea siguiente. Puedes copiar las dos líneas juntas.)

Forma B, con el mouse: abre **JetBrains Toolbox** → clic en **IntelliJ IDEA** → botón **Open** → navega a `Desktop` → `StockFlow` → `intellij` → `stockflow-backend` → **OK**.

Si IntelliJ pregunta "*Trust and Open Project?*", elige **Trust Project**.

✓ Qué deberías ver

A la izquierda, el panel **Project** muestra `stockflow-backend` con las carpetas `src` y el archivo `pom.xml`. Abajo a la derecha aparece una barra de progreso (*Importing Maven project, Indexing...*). **Espera a que desaparezca** antes de seguir: la primera vez puede tardar unos minutos.

⚠ Si falla

Si el panel muestra la carpeta `StockFlow` con `docs` y `datagrip`, abriste la carpeta equivocada: **File** → **Close Project** y repite este paso.

Paso 5 · Elegir Java 21 en IntelliJ (sólo la primera vez)

1. Menú **File** → **Project Structure...** (atajo `Ctrl+Alt+Shift+S`).
2. A la izquierda elige **Project**.
3. En el campo **SDK** debe aparecer **21**. Si dice *No SDK* u otra versión, desplégalo y elige la 21.
4. Si la 21 no está en la lista: **Add SDK** → **JDK...** y selecciona la carpeta `/usr/lib/jvm/java-21-openjdk`.
5. Pulsa **OK**.

✓ Qué deberías ver

Desaparece cualquier aviso amarillo arriba del código que diga *Project JDK is not defined*.

⚠ Si falla

Si al ejecutar aparece `invalid source release: 21` o `Unsupported class file major version`, el SDK no es el 21: repite este paso.

Paso 6 · Ejecutar StockFlowApplication

1. Pulsa `Ctrl+N`, escribe **StockFlowApplication** y pulsa **Enter**: se abre la clase principal.
2. A la izquierda de la línea `public class StockFlowApplication` hay un **triángulo verde ▶** (también está junto a `public static void main`).
3. Haz clic en el triángulo y elige **Run 'StockFlowApplication'**.
4. Abajo se abre la ventana **Run** con el registro (*log*) del arranque.

Desde la segunda vez puedes usar el botón **▶ verde de la barra superior** o el atajo `Shift+F10`.

✓ Qué deberías ver

Estas son las líneas importantes. Las copié de un arranque real de tu proyecto:

- ✓ `Tomcat initialized with port 8080 (http)` → el servidor web se preparó en el puerto 8080.
- ✓ `HikariPool-1 - Start completed.` → el backend se conectó a PostgreSQL.
- ✓ `Database: jdbc:postgresql://localhost:5432/stockflow (PostgreSQL 16.15)` → Flyway encontró la base.
- ✓ `Schema "stockflow" is up to date. No migration necessary.` → las tablas ya existen (V1, V2 y V3 aplicadas). Si la base fuera nueva, verías `Migrating schema "stockflow" to version "1 ..."`, luego 2 y 3.
- ✓ `Started StockFlowApplication in 2.878 seconds` → **¡listo! El backend está funcionando.**

⚠ Si falla

Si ves `APPLICATION FAILED TO START` o una excepción en rojo, busca el mensaje en la tabla de problemas del capítulo 7.

i Para entenderlo

No cierres esa ventana ni pulses el cuadrado rojo ■ mientras estés probando: el backend vive mientras esa ejecución esté activa. Si la detienes, todo lo que sigue deja de responder.

Paso 7 · Comprobar en el navegador que el backend responde

Abre tu navegador y entra a esta dirección:

```
http://localhost:8080/api/health
```

✓ Qué deberías ver

Un texto JSON parecido a: `{"status": "OK", "application": "stockflow-backend", "project": "StockFlow - Inventario y Movimientos de Almacén (PA-06)", ...}`. Después prueba `http://localhost:8080/api/categorias`: debe mostrar las **4 categorías** de la semilla (Material de oficina, Limpieza, Seguridad industrial, Tecnología), y `http://localhost:8080/api/productos` los **5 productos**.

⚠ Si falla

Si el navegador dice *No se puede conectar* o *Connection refused*, el backend no está corriendo: vuelve al paso 6 y revisa que diga `Started StockFlowApplication`.

Paso 8 · Probar la API con Swagger UI

Swagger es una página web que lista todos los endpoints y permite probarlos con botones.

1. Entra a:

```
http://localhost:8080/swagger-ui.html
```

2. Verás grupos como `categoria-controller`, `producto-controller`, `ubicacion-controller` e `inventario-controller`.

3. Abre por ejemplo **GET /api/productos/{id}** → botón **Try it out** → escribe `1` en `id` → botón **Execute**.

4. Más abajo aparece la respuesta: **Code 200** y el JSON del producto *Resma papel bond A4 75g*.

5. Prueba también un error: `id = 9999` → **Code 404** con el mensaje *No existe el producto con id: 9999*.

✓ Qué deberías ver

Las respuestas con su código HTTP (200, 201, 404...) y el JSON.

⚠ Si falla

Si la página no carga, revisa que el backend siga corriendo (paso 6).

Paso 9 · Probar todos los casos con requests.http

`requests.http` es un archivo con **todas las pruebas HTTP preparadas**: creación, consulta, edición, borrado, errores y CORS.

1. En IntelliJ pulsa `Ctrl+Shift+N`, escribe **requests.http** y pulsa Enter.
2. Cada petición tiene un ► **verde** a su izquierda. Haz clic: la respuesta aparece en un panel abajo.
3. **Ejecútalas en orden, de arriba hacia abajo**. Los primeros POST guardan el id creado en `{{categoriaId}}` y `{{productoId}}`, que usan luego los PUT y DELETE. Si te saltas el POST, esos pasos no saben qué id usar.
4. El título de cada petición dice el código que debe dar, por ejemplo `### [409] Código UNIQUE repetido`. Compara con lo que obtienes.

Si IntelliJ pregunta por un *environment*, elige **No Environment**.

✓ Qué deberías ver

Cada respuesta con el código que indica su título: 201, 200, 400, 404, 409, 422 o 204.

⚠ Si falla

Si un PUT o DELETE con `{{productoId}}` falla, ejecuta primero el POST que lo crea (más arriba en el archivo). Si ejecutas todo dos veces, algunos POST darán 409 porque el código ya existe: es correcto, no es un error del backend.

Paso 10 · Ver el SQL que ejecuta Hibernate

Mientras pruebas, mira la ventana **Run** del paso 6. Con cada petición aparecen las sentencias SQL que el backend envía a PostgreSQL, por ejemplo:

```
select
  cje1_0.categoria_id,
  cje1_0.activo,
  cje1_0.codigo,
  ...
```

Un POST muestra un `insert into stockflow.producto ...`, un PUT un `update stockflow.producto set ...` y un DELETE un `delete from stockflow.producto ....`

✓ Qué deberías ver

SQL con formato legible después de cada petición.

i Para entenderlo

Está activado en `application.yml` (`org.hibernate.SQL: debug` y `format_sql: true`). **Es la evidencia que te pedirán en la defensa** para demostrar que un POST llegó de verdad a PostgreSQL (pregunta 40).

Paso 11 · Ver los datos en DataGrip

1. Abre **DataGrip** (desde Toolbox).
2. Botón **+** (arriba a la izquierda) → **Data Source** → **PostgreSQL**.
3. Completa: **Host** `localhost` · **Port** `5432` · **Database** `stockflow` · **User** `stockflow_admin` · **Password** `Cambia_Esta_Clave_2026!`.
4. Si abajo aparece *Download missing driver files*, haz clic para descargarlos.
5. Pulsa **Test Connection**: debe decir *Succeeded*. Luego **OK**.
6. En el panel de la izquierda despliega `stockflow` → `stockflow` → `tables`: verás **18 tablas** (las 17 del proyecto más `flyway_schema_history`).
7. Para ejecutar las consultas preparadas: **File** → **Open** → `Desktop/StockFlow/datagrip/03_verificacion_y_pruebas.sql`. Pon el cursor en una consulta y pulsa **Ctrl+Enter**.

✓ Qué deberías ver

Los resultados en una tabla debajo del editor.

⚠ Si falla

La **sección 7** del script tiene pruebas que **deben fallar** (muestran que las constraints funcionan): un error rojo ahí es lo esperado. La **sección 8** modifica datos (atiende una solicitud); es segura de repetir, porque la segunda vez no cambia nada.

Paso 12 · Ejecutar las pruebas automáticas

1. En IntelliJ, en el panel **Project**, haz clic derecho sobre la carpeta `src/test/java`.
2. Elige **Run 'All Tests'**.
3. Se abre la ventana de pruebas con una barra que se pone verde o roja.

✓ Qué deberías ver

Barra **verde** y *Tests passed: 60*. La primera vez tarda más, porque descarga la imagen de PostgreSQL para las pruebas de integración.

⚠ Si falla

Si Docker está apagado, las 11 pruebas de `PersistenciaPostgresTest` aparecen como **omitidas** (*ignored* o *skipped*); no es un error. Enciende Docker (paso 1) y repite para que pasen las 60.

i Para entenderlo

Las pruebas no usan tu base de desarrollo: `PersistenciaPostgresTest` crea un PostgreSQL temporal en Docker (Testcontainers) y lo borra al terminar. Puedes ejecutarlas sin miedo a perder datos.

4. Poner en marcha el frontend (React), paso a paso

El frontend es la aplicación web con pantallas. Vive en `webstorm/stockflow-frontend` y se trabaja con **WebStorm**. Necesita la base y el backend del capítulo 3 **encendidos**: si no, se abre, pero no puede mostrar datos.

Vas a tener dos programas corriendo a la vez: el backend en IntelliJ (puerto 8080) y el frontend en la terminal de WebStorm (puerto 5173). El navegador abre el 5173, y el frontend le pide los datos al 8080.

Frontend 1 · Abrir el frontend en WebStorm

Antes: el backend tiene que estar corriendo (capítulo 3, paso 6, con **Started StockFlowApplication**). Déjalo encendido en IntelliJ; el frontend se abre en **otra** herramienta.

Igual que con el backend, hay que abrir la carpeta exacta: `webstorm/stockflow-frontend`, la que contiene `package.json`.

Forma A, desde la terminal:

```
~/local/share/JetBrains/Toolbox/scripts/webstorm \
~/Desktop/StockFlow/webstorm/stockflow-frontend
```

Forma B, con el mouse: **JetBrains Toolbox** → **WebStorm** → **Open** → **Desktop** → **StockFlow** → **webstorm** → **stockflow-frontend** → **OK** → **Trust Project**.

✓ **Qué deberías ver**

En el panel **Project**: `stockflow-frontend` con las carpetas `src` y `public` y los archivos `package.json`, `vite.config.ts` y `.env.development`.

⚠ **Si falla**

Si ves la carpeta `StockFlow` con `docs` e `intellij`, abriste la carpeta equivocada: **File** → **Close Project** y repite.

Frontend 2 · Instalar las dependencias (la primera vez)

Abre la terminal **dentro de WebStorm**: menú **View** → **Tool Windows** → **Terminal** (atajo **Alt+F12**). Ya se abre en la carpeta del frontend. Escribe:

```
npm install
```

✓ **Qué deberías ver**

Tarda unos segundos y termina con líneas como `added 109 packages` y `found 0 vulnerabilities`. Aparece la carpeta `node_modules` en el panel Project.

⚠ **Si falla**

Si WebStorm muestra arriba un aviso `"Run 'npm install'"`, puedes pulsar ese botón en vez de escribir el comando: hace lo mismo. Si aparece **ERR!** por la red, revisa tu conexión y repite.

i Para entenderlo

`node_modules` (las librerías descargadas) **no se sube a git** porque pesa mucho; cada integrante lo genera con `npm install`. El archivo `package-lock.json` sí está en git y garantiza que todos instalen **exactamente las mismas versiones**. Sólo hay que repetirlo cuando cambie `package.json` (por ejemplo, después de un `git pull` que agregue una librería).

Frontend 3 · Revisar la dirección del backend (.env.development)

Abre el archivo `.env.development` (está en la raíz de `stockflow-frontend`). Debe contener exactamente:

```
VITE_API_URL=http://localhost:8080/api
```

✓ Qué deberías ver

Esa única línea. **No hace falta cambiar nada**: ya apunta al backend del paso 6.

⚠ Si falla

Si el archivo no existe, créalo copiando `.env.example` (tiene la misma línea).

i Para entenderlo

El frontend no escribe la dirección del backend dentro del código: la lee de esta variable (`src/config/env.ts`). Así, para producción basta con cambiar `.env.production`. Sólo las variables que empiezan con `VITE_` llegan al navegador, por eso **nunca** se ponen contraseñas aquí: cualquiera podría verlas.

Frontend 4 · Encender el frontend

En la terminal de WebStorm:

```
npm run dev
```

✓ Qué deberías ver

Copiado de un arranque real del proyecto:

- ✓ `VITE v8.3.2 ready in 147 ms` → el servidor de desarrollo arrancó.
- ✓ `→ Local: http://localhost:5173/` → la dirección donde está la aplicación.

⚠ Si falla

Si dice `Error: Port 5173 is already in use`, ya hay otro `npm run dev` corriendo (otra terminal u otra ventana de WebStorm): ciérralo con `Ctrl+C` o búscalo con `ss -ltnp | grep 5173`. El proyecto está configurado para **no** cambiarse de puerto, porque el backend sólo acepta peticiones desde el 5173.

i Para entenderlo

Igual que con el backend: **no cierras esa terminal** mientras uses la aplicación. Cuando guardas un archivo del frontend, la página del navegador se actualiza sola (*hot reload*), sin reiniciar nada.

Frontend 5 · Abrir la aplicación en el navegador

Haz **Ctrl+click** sobre la dirección que mostró la terminal, o pégala en el navegador:

```
http://localhost:5173
```

✓ Qué deberías ver

- La página **StockFlow** con un menú azul a la izquierda (Inicio, Categorías, Productos):
 - ✓ **Resumen del inventario** con tres tarjetas: Categorías, Productos y Alertas de stock mínimo.
 - ✓ En la tabla **Stock bajo (RF-20)**: **PRD-OFI-002** · **Bolígrafo azul** en *Depósito Norte* con cantidad 25 y mínimo 100.
 - ✓ Los números salen **de la base de datos**, pasando por el backend: no están escritos en el frontend.

⚠ Si falla

Si aparece un recuadro rojo "**No se pudo conectar con el backend. ¿Está corriendo StockFlowApplication en el puerto 8080?**", el backend está apagado: vuelve al capítulo 3, paso 6, y luego recarga la página con **F5**.

Frontend 6 · Probar el CRUD desde las pantallas

Este recorrido demuestra que el frontend, el backend y la base trabajan juntos (sirve para la defensa):

1. **Categorías** → **Nueva categoría**: pulsa **Crear categoría** con todo vacío → aparece *El código es obligatorio* debajo del campo. Es la validación del frontend: **no** se envió nada al backend.
2. Escribe código **CAT-DEMO** y nombre **Demostración** → **Crear categoría** → mensaje verde *Categoría CAT-DEMO creada* y la fila aparece en la tabla.
3. Repite el mismo código → mensaje rojo *Ya existe una categoría con el valor: CAT-DEMO*. Ese texto lo escribió el **backend** (409).
4. En la fila **Material de oficina** pulsa **Eliminar** → **Eliminar** en la ventana de confirmación → mensaje rojo: *...tiene productos asociados y no se puede eliminar...* La fila **sigue** en la tabla, porque el backend rechazó el borrado.
5. **Productos**: elige la categoría **CAT-DEMO**, código **PRD-DEMO-001**, nombre **Producto demo**, stock mínimo **5** → **Crear producto**. Usa **Editar** para cambiarle el nombre y **Eliminar** para borrarlo.
6. Vuelve a **Categorías** y elimina **CAT-DEMO** (ya sin productos): desaparece de la tabla.

✓ Qué deberías ver

Mientras haces cada acción, en la ventana **Run** de IntelliJ aparecen los **insert**, **update** y **delete** que el backend ejecutó en PostgreSQL: la prueba de que los datos llegaron a la base.

⚠ Si falla

Si algo no responde, pulsa **F12** en el navegador → pestaña **Console** y busca el mensaje en rojo en la tabla de problemas del capítulo 7.

i Para entenderlo

El frontend sólo **muestra** el resultado: las reglas (código único, no borrar una categoría con productos) las decide el backend. Por eso, cuando el backend responde 409, la fila no se quita de la pantalla.

Frontend 7 · Ejecutar las pruebas y el build del frontend

En la terminal de WebStorm abre **otra pestaña** (botón +), para no detener `npm run dev`, y ejecuta:

```
npm run test:run
npm run lint
npm run build
```

✓ Qué deberías ver

Lo que muestra cada comando en tu proyecto:

- ✓ `npm run test:run` → `Test Files 8 passed (8)` y `Tests 28 passed (28)`.
- ✓ `npm run lint` → ninguna advertencia (sólo las líneas `npm notice run ...`).
- ✓ `npm run build` → `✓ built in ...ms` y la carpeta `dist/` con la versión lista para publicar.

⚠ Si falla

Si el build muestra `error TS...`, es un error de tipos de TypeScript: el mensaje indica archivo y línea. Las pruebas del frontend **no** necesitan el backend encendido: usan datos simulados.

i Para entenderlo

Las pruebas están junto al código (`*.test.ts` y `*.test.tsx`) y usan Vitest y Testing Library. También puedes ejecutarlas en WebStorm con el ► verde junto a cada `describe` o `it`.

5. Cómo apagar todo

Al terminar, apaga en este orden:

Qué apagar	Cómo	¿Se pierden datos?
El frontend	En la terminal donde corre <code>npm run dev</code> : <code>Ctrl+C</code>	No (el frontend no guarda datos)
El backend	Botón cuadrado rojo ■ en la ventana Run de IntelliJ (o <code>Ctrl+F2</code>)	No
La base de datos	<code>cd ~/Desktop/StockFlow</code> y luego <code>docker compose down</code>	No : los datos quedan guardados en un volumen de Docker
Docker Desktop	<code>systemctl --user stop docker-desktop</code>	No

Lo que NO debes hacer

- ✗ **No** uses `docker compose down -v`, salvo que quieras **borrar la base** y empezar de cero. La `-v` borra el volumen con todos los datos; al volver a arrancar el backend, Flyway recrea las tablas y la semilla original.
- ✗ **No** edites los archivos `V1__...`, `V2__...` ni `V3__...` de `db/migration`: Flyway detecta el cambio y el backend no arranca. Los cambios de base van en un archivo nuevo, `V4__descripcion.sql`.
- ✗ **No** guardes contraseñas ni tokens en archivos dentro del proyecto (como pasó con `credenciales.env`), y menos en un `.env` del frontend: todo lo que empieza con `VITE_` se publica en el navegador.
- ✗ **No** subas `node_modules` ni `dist` a git (ya están en el `.gitignore` del frontend) y **no** borres `package-lock.json`.
- ✗ **No** abras el frontend desde `http://127.0.0.1:5173` ni desde otro puerto: el backend sólo acepta `http://localhost:5173` (CORS).

6. Rutina rápida (para los días siguientes)

Cuando ya hiciste todo una vez, esto es lo único que necesitas:

Encender

- `systemctl --user start docker-desktop`
- `cd ~/Desktop/StockFlow && docker compose up -d`
- Abrir IntelliJ (`intellij/stockflow-backend`) → ► **StockFlowApplication** → esperar `Started StockFlowApplication`
- Abrir WebStorm (`webstorm/stockflow-frontend`) → terminal `Alt+F12` → `npm run dev`
- Usar la aplicación: `http://localhost:5173` (o probar la API sola en `http://localhost:8080/swagger-ui.html`)

Apagar

1. `Ctrl+C` en la terminal de `npm run dev`
2. ■ en IntelliJ
3. `docker compose down`
4. `systemctl --user stop docker-desktop`

7. Problemas frecuentes y cómo resolverlos

Busca en la columna izquierda el mensaje que te aparece (en la terminal, en la ventana Run de IntelliJ o en la consola del navegador, **F12**). Los del frontend están al final.

Lo que ves	Qué significa	Cómo solucionarlo
Unit docker.service not found	Intentaste arrancar Docker Engine, pero tienes Docker Desktop.	Usa <code>systemctl --user start docker-desktop</code> (sin sudo).
failed to connect to the docker API	Docker Desktop está apagado o todavía arrancando.	Paso 1 y espera unos segundos. Revisa también <code>docker context show</code> → debe ser <code>desktop-linux</code> .
Connection to localhost:5432 refused	El backend no encuentra la base: el contenedor no está corriendo.	Pasos 2 y 3 (<code>docker compose up -d</code> , esperar a <i>healthy</i>).
password authentication failed for user "stockflow_admin"	La contraseña del backend no coincide con la de la base.	Si creaste un <code>.env</code> con otra <code>DB_PASSWORD</code> , ponla también en IntelliJ: Run → Edit Configurations... → StockFlowApplication → Environment variables → DB_PASSWORD=tu_clave .
Web server failed to start. Port 8080 was already in use.	Ya hay otro backend corriendo (otra ejecución u otra ventana de IntelliJ).	Detén la otra ejecución con ■ . Si no la encuentras: <code>ss -ltnp grep 8080</code> muestra el proceso.
Found non-empty schema(s) "stockflow" but no schema history table	La base tiene tablas creadas a mano (DataGrip) antes de usar Flyway.	<code>docker compose down -v</code> , luego <code>docker compose up -d</code> y vuelve a ejecutar el backend. Borra los datos y Flyway recrea todo.
Validate failed: Migrations have failed validation ... Migration checksum mismatch for migration version 1	Alguien modificó una migración ya aplicada.	Deshaz el cambio: <code>git checkout -- intellij/stockflow-backend/src/main/resources/db/migration/</code> .
Schema-validation: missing table [...] o missing column [...]	Las entidades Java no coinciden con las tablas (por ejemplo, base vieja o un cambio sin migración).	Comprueba que estás en la rama Hector actualizada (<code>git pull</code>). Si la base está desactualizada: <code>docker compose down -v</code> y <code>up -d</code> .
No aparece el ► verde en StockFlowApplication	IntelliJ no cargó el proyecto Maven (abriste otra carpeta o no terminó de importar).	Revisa que abriste <code>intellij/stockflow-backend</code> . Clic derecho en <code>pom.xml</code> → Maven → Reload Project (o Add as Maven Project).
invalid source release: 21 o Unsupported class file major version	IntelliJ usa una versión de Java distinta de 21.	Paso 5 (Project Structure → SDK = 21).
Muchos errores rojos en el código (Cannot resolve symbol)	Las dependencias Maven no se descargaron todavía.	Espera a que termine la barra de progreso o pulsa el botón Reload All Maven Projects (ícono de flechas circulares en la ventana Maven, a la derecha).

Lo que ves	Qué significa	Cómo solucionarlo
El navegador dice <i>No se puede conectar a localhost:8080</i>	El backend no está corriendo.	Mira la ventana Run de IntelliJ: debe decir Started StockFlowApplication . Si no, revisa el error que muestra.
En requests.http: <i>Variable 'productid' is not defined</i>	Ejecutaste un PUT o DELETE antes del POST que guarda ese id.	Ejecuta primero el POST [201] Registrar producto con categoría existente .
Frontend: npm: command not found	Node.js y npm no están instalados.	<code>sudo pacman -S nodejs npm</code> (preparación 3).
Frontend: sh: vite: command not found	No se instalaron las dependencias (falta <code>node_modules</code>).	<code>npm install</code> en la carpeta <code>webstorm/stockflow-frontend</code> (frontend, paso 2).
Frontend: npm error enoent Could not read package.json	La terminal no está en la carpeta del frontend.	<code>cd ~/Desktop/StockFlow/webstorm/stockflow-frontend</code> y repite.
Frontend: Error: Port 5173 is already in use	Ya hay otro <code>npm run dev</code> encendido.	Ciérralo con <code>Ctrl+C</code> en su terminal; si no lo encuentras, <code>ss -ltnp grep 5173</code> muestra el proceso.
Recuadro rojo <i>No se pudo conectar con el backend...</i>	El backend (8080) está apagado o se detuvo con un error.	Capítulo 3, paso 6; luego F5 en el navegador.
En la consola del navegador (F12): <i>blocked by CORS policy: No 'Access-Control-Allow-Origin' header</i>	Abriste la aplicación desde una dirección que el backend no permite (<code>127.0.0.1</code> , otro puerto o <code>npm run preview</code> en el 4173).	Usa exactamente <code>http://localhost:5173</code> con <code>npm run dev</code> .
Página en blanco y en la consola: <i>Falta VITE_API_URL</i>	No existe <code>.env.development</code> o no tiene la variable.	Créalo desde <code>.env.example</code> (frontend, paso 3) y reinicia <code>npm run dev</code> (<code>Ctrl+C</code> y otra vez).
Cambiaste el <code>.env</code> y no pasa nada	Vite lee los <code>.env</code> sólo al arrancar.	<code>Ctrl+C</code> y <code>npm run dev</code> de nuevo.

Si el error no está en la tabla: copia el primer mensaje en rojo de la ventana Run (normalmente la línea que empieza con **Caused by:** más abajo explica la causa real) o, si es del frontend, el de la consola del navegador (**F12** → Console), y pídele ayuda a tu equipo con ese texto exacto.

8. Glosario

Término	Significado simple
Backend	El programa que procesa las peticiones y guarda los datos. No tiene pantallas: responde JSON.
Contenedor	Una "caja" donde corre un programa (aquí, PostgreSQL) aislado del resto del sistema.
Volumen	El disco de Docker donde el contenedor guarda los datos. Por eso no se pierden al apagar.
localhost	"Esta misma computadora".
Puerto	Un número que identifica a un programa dentro de la computadora: 5432 es PostgreSQL, 8080 es el backend.
Endpoint	Una dirección de la API, como <code>GET /api/productos/1</code> .
JSON	El formato de texto con que el backend envía y recibe datos: <code>{"codigo": "CAT-OFI"}</code> .
Migración	Un archivo SQL numerado (V1, V2, V3) que crea o cambia la base. Flyway los aplica en orden y recuerda cuáles ya aplicó.
Swagger UI	Una página web generada automáticamente que documenta la API y permite probarla.
Log	El registro de mensajes que el backend escribe mientras funciona (la ventana Run).
Maven	La herramienta que descarga las librerías del proyecto y lo compila. IntelliJ la usa internamente.
Frontend	La parte que el usuario ve y toca: pantallas, tablas y formularios. Se ejecuta en el navegador y le pide los datos al backend.
React	La librería con la que se construyen las pantallas del frontend, a partir de piezas llamadas componentes.
Node.js	El programa que ejecuta JavaScript fuera del navegador. Lo usan Vite, npm y las pruebas.
npm	El gestor de librerías del frontend (como Maven en el backend). Lee <code>package.json</code> .
Vite	La herramienta que sirve el frontend mientras programas (<code>npm run dev</code>) y lo empaqueta para producción (<code>npm run build</code>).
node_modules	La carpeta con las librerías descargadas por <code>npm install</code> . No se sube a git.
CORS	La regla del navegador que impide que una página llame a un servidor de otro origen, salvo que ese servidor lo permita. El backend permite sólo <code>http://localhost:5173</code> .